



University of Global Village (UGV), Barishal

Department of Computer Science and Engineering

Object Oriented Programming (Java)

Course Teacher

MD. MEHADI HASAN

Lecturer

Department of CSE

University of Global Village(UGV), Barishal

Topic 3: Class and Objects

Outline

- Class
- Object
- Constructors

Class & Objects

Objects:

Objects have states and behaviors. Example: A dog has states-color, name, breed as well as behaviors -wagging, barking, eating. An object is an instance of a class.

Class:

Class - A class can be defined as a template/blue print that describes the behaviors/states that object of its type support.

Class & Objects

- A class is a template or blueprint from which objects are created.

```
class NameOfClass
{
    constructor1        // construction of object
    constructor2
    . . .
    method1              // behavior of object
    method2
    . . .
    field1               // state of object
    field2
    . . .
}
```

Class & Objects/ Example

Creating a class:

```
class Box {  
    double width;// Instance variable  
    double height;  
    double depth;  
}
```

Creating an object:

```
Box mybox = new Box(); // create a Box object called mybox
```

Access instance variables:

Use the dot(.) operator.

```
mybox.width = 100;
```

Class & Objects/ Example

```
class Box {  
    double width;  
    double height;  
    double depth;  
}
```

```
class BoxDemo {  
    public static void main(String args[]) {  
        Box mybox = new Box();  
        double vol;  
  
        mybox.width = 10;  
        mybox.height = 20;  
        mybox.depth = 15;  
  
        vol = mybox.width * mybox.height * mybox.depth; // compute volume of box  
        System.out.println("Volume is " + vol);  
    }  
}
```

Class & Objects/ Example

For two Box objects, each has its own copy of depth, width, and height

```
class Box {  
    double width;  
    double height;  
    double depth;  
}  
  
class BoxDemo2 {  
    public static void main(String args[]) {  
        Box mybox1 = new Box();  
        Box mybox2 = new Box();  
        double vol;  
  
        mybox1.width = 10;  
        mybox1.height = 20;  
        mybox1.depth = 15;
```


Class & Objects/ Example

For two Box objects, each has its own copy of depth, width, and height

```
mybox2.width = 3;
```

```
mybox2.height = 6;
```

```
mybox2.depth = 9;
```

```
vol = mybox1.width * mybox1.height * mybox1.depth;
```

```
System.out.println("Volume is " + vol);
```

```
vol = mybox2.width * mybox2.height * mybox2.depth;
```

```
System.out.println("Volume is " + vol);
```

```
}
```

```
}
```

Volume is 3000.0

Volume is 162.0

Class & Objects/ Example

Adding a Method to the Box Class:

```
class Box {  
    double width;  
    double height;  
    double depth;  
  
    // display volume of a box  
    void volume() {  
        System.out.print("Volume is ");  
        System.out.println(width * height * depth);  
    }  
}
```

Class & Objects/ Example

Returning a Value:

```
class Box {  
    double width;  
    double height;  
    double depth;  
  
    // compute and return volume  
    double volume() {  
        return width * height * depth;  
    }  
}
```

Class & Objects/ Example

Adding a Method That Takes Parameters:

```
class Box {  
    double width;  
    double height;  
    double depth;  
  
    // compute and return volume  
    double volume() {  
        return width * height * depth;  
    }  
    int square(int i)  
    {  
        return i * i;  
    }  
}
```

Constructors

- A class can have one or more constructors.
- If we do not explicitly write a constructor for a class the Java compiler builds a default constructor for that class.
- It has the same name as the class in which it resides and is syntactically similar to a method.
- They have no return type, not even void.
- Almost always public, although can be others.

Once you define your own constructor, the default constructor is no longer used.

Constructors/ No parameter

```
class Box {  
    double width;  
    double height;  
    double depth;  
  
    // This is the constructor for Box.  
    Box() {  
        System.out.println("Constructing Box");  
        width = 10;  
        height = 10;  
        depth = 10;  
    }  
  
    // compute and return volume  
    double volume() {  
        return width * height * depth;  
    }  
}
```

Constructors/ parameterized

- Parameterized Constructors

```
class Box {  
    double width;  
    double height;  
    double depth;  
  
    Box(double w, double h, double d) {  
        width = w;  
        height = h;  
        depth = d;  
    }  
  
    double volume() {  
        return width * height * depth;  
    }  
}
```

Constructors/ this keyword

- **this** can be used inside any method to refer to the current object

```
class Box {  
    double width;  
    double height;  
    double depth;  
  
    // A redundant use of this.  
    Box(double w, double h, double d) {  
        this.width = w;  
        this.height = h;  
        this.depth = d;  
    }  
  
    double volume() {  
        return width * height * depth;  
    }  
}
```


Constructors/ this keyword

- Instance Variable Hiding:

When a local variable has the same name as an instance variable, the local variable hides the instance variable.

// Use this to resolve name-space collisions.

```
Box(double width, double height, double depth) {  
    this.width = width;  
    this.height = height;  
    this.depth = depth;  
}
```

```

class ThisKeyword {
    double width;
    double height;
    double depth;
    // This is the constructor for Box.
    ThisKeyword(double width, double height, double depth) {
        width = width;
        height = height;
        depth = depth;
    }
    ThisKeyword(double width, double height, double depth, boolean flag) {
        this.width = width;
        this.height = height;
        this.depth = depth;
    }
    public static void main (String[] args) {
        ThisKeyword tk = new ThisKeyword(10,10,1);
        System.out.println(" "+tk.width+" "+tk.height+" "+tk.depth);

        ThisKeyword thisK = new ThisKeyword(100,100,100,true);
        System.out.println(" "+thisK.width+" "+thisK.height+" "+thisK.depth);
    }
}

```

0.0 0.0 0.0
100.0 100.0 100.0

Enough For Today

Thanks all of you.